

Protocolo de recuperação colaborativa em Replicação Máquina de Estados

Niccolas Guntzel Maganeli, Fernando Dotti

¹Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)
90.619-900 – Porto Alegre – RS – Brazil

niccolas.maganeli@edu.pucrs.br, fernando.dotti@pucrs.br

Abstract. *This article proposes a collaborative protocol for replica recovery, within a distributed system based on replicated state machines, with the objective of reducing recovery time and impacts on system throughput during the procedure. In addition to contextualizing and motivating this subject, this work presents a functional prototype, experiments and results of the application of this technique.*

Resumo. *Este artigo propõe um protocolo colaborativo para a recuperação de réplicas, dentro de um sistema distribuído baseado em máquinas de estado replicadas, com o objetivo de reduzir o tempo de recuperação e os impactos na vazão do sistema durante o procedimento. Além de contextualizar e motivar este assunto, este trabalho apresenta um protótipo funcional, experimentações e resultados da utilização desta técnica.*

1. Introdução

Serviços online necessitam, cada vez mais, serem altamente disponíveis, ou seja, na maior parte do tempo estarem online e provendo o serviço. Seja por sua funcionalidade, ou para atingir alta disponibilidade, estes serviços são modelados de forma distribuída. A medida que dependemos de tais serviços para realizar tarefas mais importantes, seja economicamente ou devido a funções nas quais confiamos parte do nosso trabalho, por exemplo, estes sistemas se tornam mais críticos e qualquer falha pode trazer prejuízos para as diversas partes envolvidas. Dito isso, estes serviços devem ser capazes de suportar falhas, ou seja, serem capazes de continuar provendo, na presença de falhas parciais, que podem estar relacionadas a diversos fatores: problemas físicos, *bugs* na implementação do serviço, comportamentos maliciosos injetados por terceiros, etc.

Uma das técnicas existentes para adicionar tolerância a falhas a uma aplicação é através de replicação máquinas de estados (RME ou SMR, do inglês, State-Machine Replication) [Lamport 1978, Schneider 1990]. A técnica de SMR é bastante simples e pode ser descrita de acordo com os seguintes passos: i) réplicas iniciam no mesmo estado; ii) recebem a mesma sequência de requisições; iii) processam estas requisições de forma determinística (dado um estado e uma requisição, só existe um estado resultante possível). Desta forma, caso alguma das réplicas venha a falhar, o sistema ainda consegue prover o serviço, dado um limite de réplicas falhas. Sistemas distribuídos são, normalmente, desenhados em questão de servidores e clientes [Schneider 1990]. Fazendo com que a abordagem de SMR seja uma combinação bastante adequada para, possivelmente, solucionar este problema. A estrutura mais simples que essas réplicas podem apresentar é

composta pelos seguintes elementos: i) um registro com as operações executadas, chamado de *log*; ii) uma série de imagens do estado construídas em determinados momentos, chamados de *checkpoints*; iii) estado atual da réplica. Os *checkpoints* servem para que o registro de operações não cresça indeterminadamente (podendo ocupar todo o disco rígido do servidor) e para que a transmissão de estado possa ser realizada.

Falhas vão acontecer e, para que o sistema consiga manter o nível de tolerância a falhas, uma outra réplica precisa ser inserida no sistema. Este processo é chamado de recuperação. A recuperação implica na transferência de estado entre as réplicas. O estado enviado de uma réplica para outra, no contexto da recuperação, envolve: i) o último *checkpoint* realizado por todas as réplicas (em consenso); ii) *logs* com as operações realizadas desde o *checkpoint* sendo transmitido e que devem ser executadas pela nova réplica [Bessani et al. 2013]. Esta transferência pode ser volumosa, impactando o tempo de transferência e a vazão do sistema.

Na abordagem mais simples de recuperação, tanto a réplica se recuperando quanto a réplica transferindo o estado não estão provendo o serviço, consequentemente gerando um impacto na vazão do sistema. Por exemplo, num sistema com $f = 1$, com 3 réplicas, onde f é o número de réplicas falhas permitidas, a recuperação mantém somente 1 réplica provendo o serviço. Além de permitir reinserir uma réplica faltante, os mecanismos de recuperação possibilitam também a manutenção do sistema replicado sem interrupções do serviço prestado - o que é igualmente importante a tolerar falhas. Esta funcionalidade é importante, por exemplo, para redimensionar sistemas ou mesmo migrar a localização de sistemas sem interrupção dos serviços, assim como sincronizar estado em configurações onde nodos (réplicas) podem entrar (e sair) sem um controle central (como no exemplo de blockchains não permissionadas).

Este trabalho propõe um protocolo de recuperação colaborativo, onde mais de uma réplica ativa participa do processo de transferência de estado. A motivação por trás da criação deste protocolo é medir o impacto da divisão do trabalho entre mais de uma réplica. Também será apresentando um protótipo construído com a ferramenta BFT-SMaRt [Bessani et al. 2014]. Para fins de analisar os resultados, a abordagem colaborativa proposta neste trabalho será comparada com a abordagem clássica, onde uma réplica sozinha é responsável por toda a transferência.

O resto do artigo está organizado da seguinte forma: Seção 2 apresenta trabalhos relacionados, Seção 3 apresenta o protocolo de recuperação proposto, Seção 4 apresenta os experimentos realizados juntamente com a análise e, por fim, Seção 5 apresenta as conclusões e trabalhos futuros.

2. Trabalhos relacionados

Máquinas de estado replicadas vem sendo utilizadas há bastante tempo para o gerenciamento de serviços distribuídos [Schneider 1990]. Estas máquinas de estado possuem propriedades interessantes para que o sistema funcione corretamente e os estados gerados possam ser compartilhados entre as réplicas. Todas as réplicas ativas precisam receber todos os comandos e estas devem concordar na ordem em que os comandos são executados. Disto surge a principal restrição desta técnica de replicação: o determinismo. Desta forma, com uma sequência de requisições e N réplicas, sempre teremos o mesmo estado final em cada uma delas e, quando necessário, podemos facilmente decidir o que deve

ser transferido para uma réplica sendo recuperada. Para que as réplicas consigam decidir a ordem de recebimento e execução dos comandos, é necessário que haja um consenso entre elas. Dependendo do tipo de falhas que se deseja suportar, o nível de replicação pode mudar. Existem dois tipos clássicos de falhas: benignas e bizantinas. Um sistema que suporta falhas benignas necessita que N réplicas estejam ativas, onde $N = 2f + 1$, e, para suportar falhas bizantinas, $N = 3f + 1$ [Cachin et al. 2011]. Em ambos os casos, f é o número máximo de réplicas falhas suportado. Para este trabalho, arbitrariamente não serão consideradas falhas bizantinas.

Outro conceito importante para a recuperação, presente em replicação de máquinas de estado, são os *checkpoints* e *logs*. O tamanho de ambos impacta a transferência de estado. Os *checkpoints* ajudam a minimizar o tamanho dos *logs* e ambos devem estar presentes em uma transferência bem sucedida. Para recuperar uma réplica, faz-se necessário instalar o estado recebido das demais e se atualizar conforme o *log*, que ainda não consta no último *checkpoint* - ou seja, processar os comandos do *log* recebido sobre o novo estado instalado. Para que uma réplica operante possa fornecer estes dados a outra em recuperação, durante a execução normal de uma réplica (sem falhas), ela é responsável por registrar requisições processadas no seu registro e de tempos em tempos deve gerar *checkpoints* [Mendizabal et al. 2017], consequentemente reduzindo o tamanho *log* neste momento.

A abordagem tradicional de transferência de estado entre máquinas replicadas ocorre da seguinte forma: i) uma nova réplica solicita o último *checkpoint* e o *log* para as demais réplicas no sistema; ii) uma das réplicas ativas é selecionada e fica responsável por enviar ambos. Durante todo o processo de transferência, a réplica ativa responsável não é capaz de manter o provimento do serviço em sua totalidade, consequentemente reduzindo a vazão do sistema durante toda a recuperação da nova réplica, que também não provê o serviço ainda. Em [Bessani et al. 2013], é apresentada uma abordagem que visa maximizar a performance das réplicas em três frentes diferentes. Temos: i) o registro de operações de forma paralela; ii) *checkpoints* realizados em sequência entre as réplicas; iii) uma abordagem "colaborativa" para transferência do estado. Para este trabalho, a última das três abordagens é a mais interessante, pois apresenta uma outra forma de transferência.

A abordagem colaborativa proposta por [Bessani et al. 2013] funciona da seguinte forma: i) uma réplica fica responsável por enviar o estado inteiro; ii) as demais réplicas enviam partes diferentes do *log* (e.g.: parte superior e parte inferior). Ainda que haja uma redução no tamanho das mensagens trocadas entre as réplicas, a colaboração empregada se limita somente aos *logs* e existe um limite máximo de réplicas que podem colaborar nesta divisão. Como os *checkpoints* são empregados para que o *log* se mantenha o menor possível, seria interessante adicionar esta colaboração também na transferência do estado, dado que este não pode ser reduzido durante o tempo de vida de uma réplica (somente se forem realizadas operações externas que reduzam). O protocolo proposto neste trabalho faz exatamente isso: além dos *logs* enviados de forma colaborativa, o estado também é particionado e todas as demais réplicas ativas são envolvidas, independente de quantas forem.

O protocolo de recuperação colaborativo proposto é construído em cima do BFT-SMaRt. Esta é uma ferramenta que constrói um sistema distribuído com base em um universo de processos que podem ser divididos em réplicas e clientes [Bessani et al. 2014].

São construídas diversas classes e interfaces que, quando juntas, são utilizadas para realizar a comunicação entre os diversos sistemas. Réplicas dentro da ferramenta mantêm o seu *log* atualizado com um lote de requisições e, periodicamente, geram *checkpoints*. A operação relevante para o desenvolvimento do protocolo ocorre quando algum dos seguintes cenários é detectado: i) uma réplica falha e é reiniciada; ii) uma réplica detecta que está mais lenta que as demais; iii) as réplicas precisam ser sincronizadas; iv) uma nova réplica é adicionada ao sistema. A partir da detecção de qualquer um dos eventos listados anteriormente, é iniciado o processo de recuperação, onde a réplica se recuperando envia uma solicitação *STATE_REQUEST* para todas as demais. A medida que as demais réplicas recebem esta solicitação, elas respondem com uma mensagem *STATE_REPLY* contendo a versão do estado solicitado. Quando a réplica que está se recuperando recebe todas as respostas com as informações desejadas, é iniciada a etapa de *STATE_DELIVER*. A réplica instala o *checkpoint* recebido e processa os *logs* que ainda não foram persistidos. O protocolo padrão de transferência, dentro da ferramenta BFT-SMaRt, envia o estado e o *log* a partir de uma só réplica. Caso a réplica selecionada não consiga enviar o estado, é sorteada uma nova réplica. Este processo pode ocorrer indefinidamente, pois não existe detecção de falhas e a (nova) réplica selecionada pode não estar mais ativa no sistema.

3. Recuperação colaborativa

Um sistema, desenvolvido em cima da ferramenta BFT-SMaRt, é formado por um conjunto finito de servidores $S = \{S_1, S_2, \dots, S_n\}$ e um conjunto variável de clientes $C = \{C_1, C_2, \dots\}$. Cada um dos servidores, internamente, possui três propriedades: i) A_n , é o estado atual do servidor S_n ; ii) L_n , é o *log* atual do servidor S_n ; iii) Ch_n , é o último *checkpoint* do servidor S_n . Para que o sistema possa se recuperar, é necessário que $failedReplicas(S) \leq f$, onde $failedReplicas$ é uma função que determina o número de réplicas falhas em um sistema, para qualquer sistema. Assume-se que as réplicas ativas pertencentes a S possuem o mesmo estado atual, o mesmo *log* e os mesmos *checkpoints*. Desta forma, podemos solicitar para qualquer uma delas o estado atual e, então, recuperar uma nova réplica. O protocolo proposto visa reduzir o tempo de transferência de uma réplica, dividindo o trabalho entre todas as ativas. Com o trabalho entre as réplicas dividido, é gasto menos tempo transferindo o estado e a réplica volta a prover o serviço antecipadamente. Como estamos somente trabalhando com falhas benignas ($N = 2f + 1$), para fins de facilitar a explicação do funcionamento do protocolo, assumiremos $f = 1$, ou seja, $N = 3$ e $S = \{S_1, S_2, S_3\}$.

3.1. Protocolo de recuperação colaborativa

A recuperação colaborativa inicia quando a réplica falha/nova solicita o estado para as demais (assumiremos que a réplica falha é a S_3) e conforme os seguintes passos: i) solicitação do estado partindo da réplica a ser recuperada; ii) cada réplica ativa deve particionar o *checkpoint* solicitado juntamente com o *log* atual. Depois de particionados ambos (e.g.: estado com 300kb, a réplica S_1 fica responsável por enviar os primeiros 150kb e a réplica S_2 fica responsável pelo resto. O mesmo ocorre para o *log*); iii) réplica S_3 espera o recebimento de todas as mensagens; iv) após o recebimento, a réplica S_3 ordena as respostas de acordo com o identificador de cada réplica, remonta o estado e os *logs*; v) réplica S_3 então instala o novo estado e processa o *log* remontado que recebeu. O pseudo código de cada uma das etapas é apresentado no Algoritmo 1, desde a requisição

de recuperação, geração do estado a ser enviado e remontagem das partes recebidas por uma réplica.

Na linha 8 é iniciada a recuperação de uma réplica. É solicitado, a todas demais, o estado do sistema a partir de um *checkpoint*, dado por *CID* (*Consensus Identifier*). No momento em que a réplica é inserida no sistema, antes mesmo da recuperação iniciar, as réplicas sincronizam entre si qual o último *CID* que está em andamento e, então, a nova réplica solicita para as demais conforme o que foi decidido. Se, durante o processo de recuperação, alguma réplica falhar em responder a requisição inicial com a parte que ficou responsável por gerar (detectado pela expiração do *timer*, na linha 13), é feita a requisição dos pedaços faltantes para as demais réplicas. Descobrir quais réplicas não responderam é bastante direto: com o conjunto R temos todas as réplicas dentro de um sistema e, através do mapa de *Replies*, temos todas as que responderam. Realizando a seguinte subtração: $R - Replies - Rid$, temos todas as réplicas que não responderam e que são diferentes de si próprias. Assim, podemos solicitar para as demais estes pedaços faltantes.

O processo de geração de estado se inicia na linha 17. A réplica recebe informações de qual *CID* deve ser remontado, qual a réplica solicitando e quais partes devem ser remontadas (partes pois aqui pode ocorrer a compensação pela falta de resposta de outras réplicas), representado por *Missing*. Nas linhas 20 e 21 são realizados procedimentos bastantes similares. O estado St é dividido em $|R| - 1$ partes e cada uma das partes solicitadas, presentes em *Missing*, é selecionada durante a divisão. Desta forma, temos dois vetores com todas as partes que devem ser geradas. Para facilitar a troca de mensagens, as partes, tanto de estado quanto as de *logs*, são acumuladas como um tripla, no seguinte formato: $(i, StPart_i, LgPart_i)$. No primeiro momento a réplica sempre é responsável por montar somente o seu pedaço de estado. Se as demais falharem, o processo é repetido para montagem das partes faltantes.

A remontagem, iniciada na linha 24, depende da resposta de todas as réplicas envolvidas no processo e só é iniciada quando todas respondem as solicitações com estados válidos. Cada mensagem recebida, registra no mapa de resposta quais partes e *logs* foram recebidos. Assim que a réplica possui todas as informações necessárias para remontar o estado (linha 28, é iniciado a última etapa do processo de recuperação. Como cada parte tem um número associado (que indica sua posição relativa às demais) for ordenada, teremos o estado completo sequencialmente pronto para ser concatenado e então atualizado. Caso não tivéssemos informações sobre onde qual parte deve ser inserida, não conseguiríamos gerar um estado correto e a recuperação colaborativa não seria possível.

3.2. Implementação

A ferramenta BFT-SMaRt foi desenhada e construída para ser bastante flexível e, por causa disso, o processo de implementação de novos serviços/protocolos é bastante simples. Analisando toda a hierarquia de classes e as etapas necessárias para adicionar o novo protocolo, podemos simplificar a estrutura da ferramenta para o modelo apresentado na Figura 1. Dentro desta arquitetura simplificada, somente é necessário o desenvolvimento de cinco primitivas: i) *getSnapshot*; ii) *installSnapshot*; iii) *requestState*; iv) *SMRequest-Deliver*; v) *SMReplyDeliver*. As duas primeiras primitivas precisavam ser implementadas na primeira camada da arquitetura, responsável pelo gerenciamento do estado e do

Algoritmo 1 Requisição, geração e remontagem do estado

```
1: Variáveis:  
2:   CID: Último checkpoint a ser requisitado  
3:   R: Conj. com todas as réplicas  
4:   Rid: Identificador da réplica  
5:    $\Delta$ : Tempo de timeout  
6:   St: Estado completo da réplica  
7:   Replies: Mapa com identificadores e partes de estado das outras réplicas  
  
8: on  $\langle \text{Recovery} \rangle$ :  
9:   forall id in  $R - \{Rid\}$  :  
10:     sendto(id, Request, CID, Rid, {id})    ▷ Solicita o estado das demais réplicas  
11:   end forall  
12:   start(timer,  $\Delta$ )  
  
13: on timer timeout:  
14:   forall id in Replies:  
15:     sendto(id, Request, CID, Rid,  $R - Replies - \{Rid\}$ )  
16:   end forall  
  
17: on  $\langle \text{Request}, CID, Source, Missing \rangle$ :    ▷ Monta sua parte e/ou de outras réplicas  
18:   Ckp  $\leftarrow$  getCkp(St, CID)  
19:   Log  $\leftarrow$  getLog(St)  
20:   StParts  $\leftarrow$  split(Ckp,  $|R| - 1$ , Missing)  
21:   LgParts  $\leftarrow$  split(Log,  $|R| - 1$ , Missing)  
22:   Res  $\leftarrow$  zip(Missing, StParts, LgParts)    ▷ Agrupa partes de estado e log  
    similares  
23:   sendto(source, Reply, Res)  
  
24: on  $\langle \text{Reply}, Res \rangle$ :  
25:   forall (i, StParti, LgParti) in Res:  
26:     put(Replies, i, StParti, LgParti)    ▷ Reg. o receb. de todas as partes e logs  
27:   end forall  
28:   if  $|Replies| == |R| - 1$  then    ▷ Verif. se recebeu todas as partes  
29:     stop(timer)  
30:     sort(replies)    ▷ Baseado nas chaves  
31:     tmpSt  $\leftarrow$  concat([], replies)    ▷ Concat., lado a lado, partes e logs ordenados  
32:     update(St, tmpSt)  
33:   end if
```

processamento das requisições. A implementação é bastante simples: basta serializar o estado quando solicitado e deserializá-lo no momento da instalação. As demais primitivas, implementadas na terceira camada, são estritamente relacionadas ao protocolo de recuperação e, para o trabalho em questão, são muito mais relevantes.

O processo de solicitação (*requestState*) pode ocorrer em dois momentos: i) quando a réplica é reiniciada/adicionada ao sistema; ii) quando as solicitações anteriores expiram. No primeiro momento, todas as demais réplicas ativas são destinatários válidos e as mensagens são enviadas para estas. Porém, no segundo momento, somente as réplicas que responderam são envolvidas no processo. Isso se deve ao fato que a ferramenta não possui uma detecção de falhas, ou seja, não é possível detectar em dado momento quais réplicas estão ativas ou quais réplicas não estão. Durante as etapas iniciais de desenvolvimento do protocolo, caso alguma réplica falhasse em enviar a parte pela qual é responsável, o processo de recuperação era reiniciado do zero. Porém, ao fazer isso, podemos acabar nunca recuperando a réplica. Desta forma, a abordagem que, após expirado o tempo de resposta das demais réplicas, se comunica com as réplicas que responderam para que o processo de recuperação se mantenha ativo.

A lógica principal do protocolo fica localizada nas primitivas *SMRequestDeliver* e *SMReplyDeliver*, ambas implementadas na terceira camada da arquitetura simplificada. A primeira primitiva acontece nas demais réplicas (diferentes da que solicitou). Todo o processo de serialização do *checkpoint*, *logs* e particionamento de ambos acontece nesta chamada. As réplicas decidem quais partes mandar de acordo com a mensagem recebida (existe um parâmetro específico com o identificador da parte "faltante"). A segunda primitiva acontece na réplica que solicitou para as demais o estado. Se tudo correu corretamente, após $N - 1$ mensagens, onde N é o número de réplicas no sistema, é iniciado o processo de recuperação. As mensagens recebidas são ordenadas em relação ao identificador, os pedaços de *checkpoint* e *logs* são concatenados sequencialmente, como descrito anteriormente e, no fim, a réplica instala o novo estado remontado e processa os logs. O código fonte com todo o desenvolvimento do protocolo pode ser encontrado em [Maganeli and Dotti 2022].

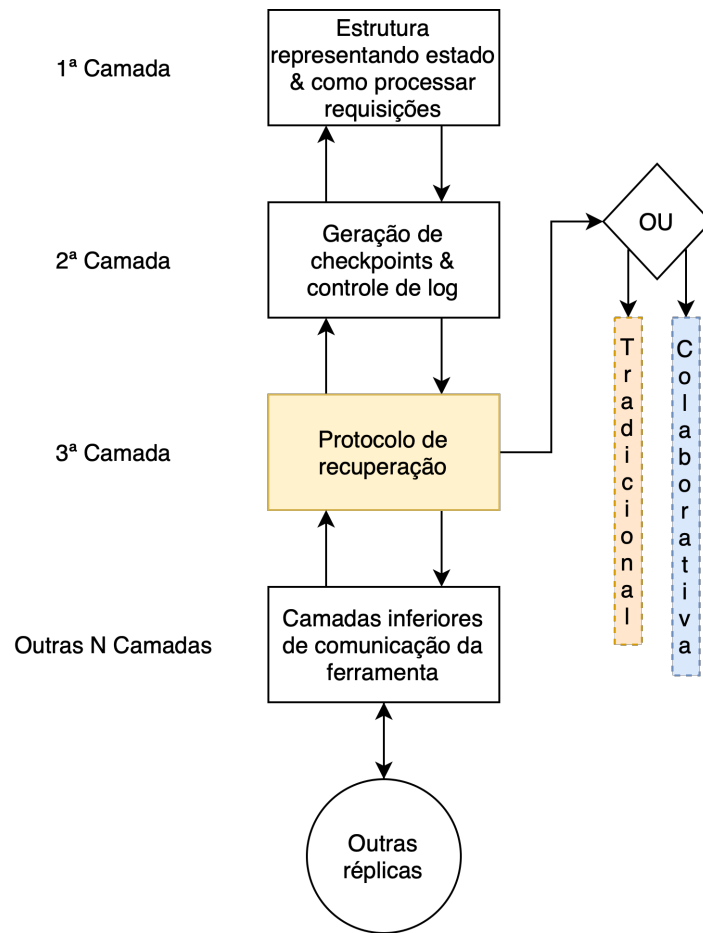


Figura 1. Arquitetura simplificada da ferramenta BFT-SMaRt

4. Experimentos

A performance da recuperação foi medida experimentalmente em um *cluster* proprietário. Este ambiente é composto de nodos com diversas tipos de configuração. Para garantir a tolerância à falha, o experimento foi executado com $f = 1$, ou seja, $N = 3$. Dentro do *cluster*, cada servidor possuía seu nó dedicado e os clientes estavam todos concentrados em um nó. Ambos, clientes e servidores, possuem a seguinte configuração: i) 2x Intel Xeon L5420 com 4 núcleos cada; ii) 120GB de SSD; iii) 8GB de memória RAM. Para verificação do impacto do protocolo na transferência de estado foram executados dois cenários distintos de testes. O primeiro envolve a transmissão de um estado com 1GB e o segundo 5GB. Estes tamanhos foram selecionados para maximizar o impacto das transferências de estado. Para realizar os experimentos foi utilizado a ferramenta YCSB [Cooper et al. 2010] que carrega um estado com um tamanho desejado e realiza diversas operações para medir a vazão do sistema. Todas as operações presentes no teste de vazão são operações de *update*, pois de acordo com [Bessani et al. 2013] são o tipo de operações mais intensas para um sistema. Desta forma o sistema é estressado em todas as frentes: transferência de estado e vazão durante a recuperação. Cada uma das baterias de experimentos foi executada cinco vezes, visando reduzir o ruído de uma execução para a outra.

A Figura 2 apresenta a comparação com duas execuções lado a lado com cada

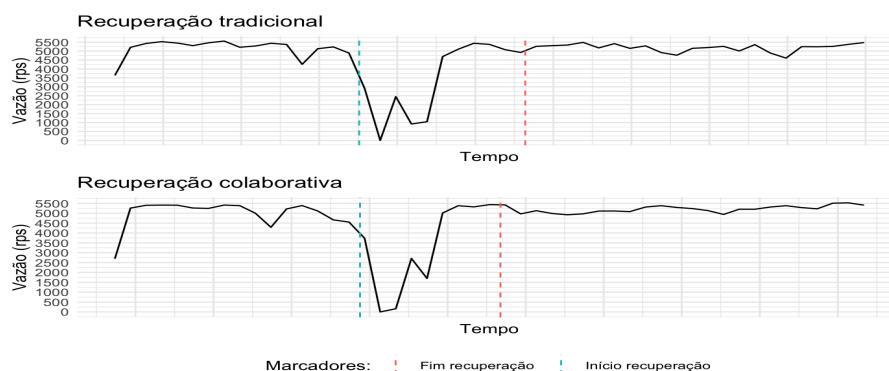


Figura 2. Comparação entre recuperação tradicional e colaborativa com estado de 1GB

um dos protocolos. É possível notar um pequeno ganho no tempo de recuperação que favorece o protocolo colaborativo proposto. Também, na Figura ??, vemos o mesmo comportamento se repetindo. O protocolo colaborativo geralmente termina o processo de recuperação antes do protocolo tradicional. Porém, não podemos nos basear somente nos gráficos apresentados, pois eles englobam somente uma execução por protocolo cada. Analisando os dados na Tabela 2, que apresenta os tempos médios e desvio padrão da recuperação, vemos que de fato o protocolo colaborativo sempre apresenta vantagens. E, se compararmos diretamente os tempos médios entre a recuperação com estados de 1GB e 5GB, vemos que o protocolo proposto melhora conforme o tamanho do estado aumenta. Podemos concluir que o envio de duas metades menores é mais vantajoso que o envio de um estado inteiro. A principal vantagem da recuperação acelerada de uma réplica é a volta aos padrões de tolerância a falhas proposto na modelagem do sistema. Fazendo com que o sistema "permita" que outra réplica possa falhar antes e, eventualmente, ser recuperada.

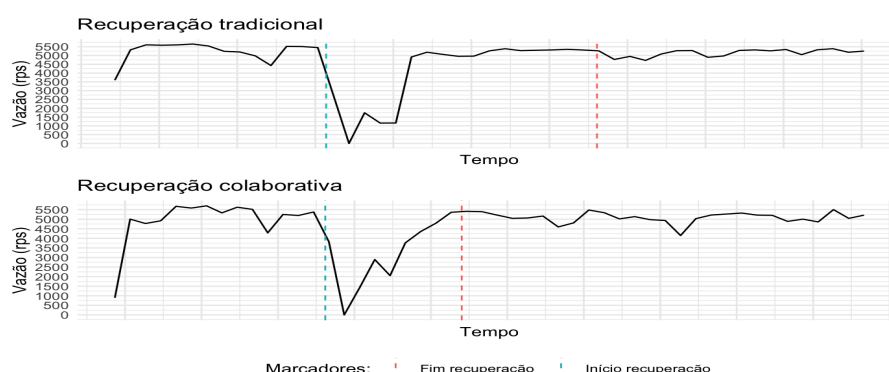


Figura 3. Comparação entre recuperação tradicional e colaborativa com estado de 5GB

A Tabela 1 apresenta os valores médios e desvio padrão da vazão do sistema durante a recuperação em ambos os protocolos. Juntamente com a análise dos gráficos, podemos ver que a média de vazão é bastante similar entre os protocolos empregados. O impacto maior, na vazão, se deu no início do processo de recuperação de cada uma das

Tabela 1. Médias e desvios padrão para cada um dos protocolos e tamanhos de estado.

Tamanho do Estado	Protocolo de recuperação	Média (rps)	Desvio padrão
1GB	Tradicional	4785.41	1228.84
	Colaborativo	4780.53	1236.94
5GB	Tradicional	4794.22	1250.18
	Colaborativo	4702.00	1213.44

Tabela 2. Médias e desvios padrão para o tempo de recuperação de cada protocolo e tamanho de estado.

Tamanho do Estado	Protocolo de recuperação	Média (s)	Desvio padrão
1GB	Tradicional	14.10	3.36
	Colaborativo	12.80	4.10
5GB	Tradicional	14.72	4.27
	Colaborativo	9.85	1.55

execuções e, isso é bastante lógico, pois as demais réplicas precisam parar o seu processamento e enviar as informações necessárias para que a outra réplica se recupere. Podemos perceber que, mesmo a proposta colaborativa alcançando a vazão máxima antes, a média das execuções e manteve bastante homogênea. Assim, podemos concluir que não houve ganhos significativos no quesito vazão ao utilizar o protocolo proposto.

5. Conclusão

Este trabalho propõe a recuperação colaborativa em RME com vistas a diminuir o tempo de sincronização de uma réplica em recuperação. Para tal, um algoritmo foi proposto e implementado sobre uma plataforma para RME, sendo seu desempenho avaliado com um primeiro conjunto de experimentos. A recuperação colaborativa proposta apresentou ganhos no tempo de recuperação de uma réplica. Nos casos de estado com 1GB e 5GB, o processo foi respectivamente 10% e 33% mais rápido se comparado ao tradicional. A vazão do sistema durante a recuperação mostra leve impacto. Experimentos adicionais são necessários para avaliar se esta variação de vazão deve-se à técnica ou se se trata de diferenças amostrais.

Para trabalhos futuros, seria interessante verificar o desempenho para um intervalo maior de tamanhos de estado e número de réplicas. Isso permitiria avaliar quando a técnica passa a apresentar ganho com relação à sequencial, até que ponto este ganho aumenta, e se existe um ponto de saturação onde não existe ganho para estado maior ou para número maior de réplicas. Também seria interessante a adição da capacidade de tolerar falhas bizantinas, dado que o protocolo proposto ainda não possui esta funcionalidade. Desta forma, a técnica colaborativa pode ser utilizada em ambientes mais críticos.

Este trabalho surgiu inicialmente da análise do impacto na vazão do sistema [Pintor and Dotti 2021], com a utilização de estruturas concorrentes apresentadas em [Prokopec et al. 2012]. Porém, no decorrer do desenvolvimento, percebemos que os protocolos de recuperação, da forma como são modelados na ferramenta BFT-SMaRt, são alheios a representação do estado. Logo, a combinação das tuas técnicas pode trazer ganhos ainda maiores para o sistema como um todo, visto que ambos trabalhos apresentam

melhorias em frentes diferentes: este no tempo médio de recuperação de uma réplica e o outro na redução do impacto da criação de *checkpoints* na vazão do sistema.

Referências

- Bessani, A., Santos, M., Felix, J., Neves, N., and Correia, M. (2013). On the efficiency of durable state machine replication. In *2013 {USENIX} Annual Technical Conference ({USENIX}{ATC} 13)*, pages 169–180.
- Bessani, A., Sousa, J., and Alchieri, E. E. (2014). State machine replication for the masses with bft-smart. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 355–362. IEEE.
- Cachin, C., Guerraoui, R., and Rodrigues, L. (2011). *Introduction to Reliable and Secure Distributed Programming*. Springer Publishing Company, Incorporated, 2nd edition.
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., and Sears, R. (2010). Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154.
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.
- Maganeli, N. G. and Dotti, F. (2022). Collaborative protocol based on BFT-SMaRt implementation. <https://github.com/NiccoMaganeli/library/tree/workingBFT>.
- Mendizabal, O. M., Dotti, F. L., and Pedone, F. (2017). High performance recovery for parallel state machine replication. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, pages 34–44.
- Pintor, E. and Dotti, F. (2021). Smarttrie: Reducing checkpoint’s impact in smr systems with a ctrie data structure. In *Anais do XXXIX Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 616–629, Porto Alegre, RS, Brasil. SBC.
- Prokopec, A., Bronson, N. G., Bagwell, P., and Odersky, M. (2012). Concurrent tries with efficient non-blocking snapshots. In *Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming*, pages 151–160.
- Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys (CSUR)*, 22(4):299–319.